DEPARTMENT OF
COMPUTER SCIENCE & ENGINEERING
Discover. Learn. Empower.

Experiment No.: 4

Student Name: Probal Dhali **UID:** 24BAI71013
Branch: AIT-CSE **(AI ML)** **Section/Group:** B
Semester: 5TH **Date of Performance:**
Subject Name: Competitive Coding – II **Subject Code:** 24CSP-305

Experiment No: 4.1

Aim:
To check whether a singly linked list is a palindrome.

Objective:

1. To understand palindrome checking in a Linked List.
2. To find the middle of the Linked List.
3. To reverse the second half of the Linked List.
4. To compare both halves efficiently.

Problem Statement:
Given the head of a singly linked list, return true if the linked list is a palindrome; otherwise, return false.

LeetCode Link: https://leetcode.com/problems/palindrome-linked-list/?utm_source=chatgpt.com

Procedure/Algorithm/Pseudocode:

1. Use slow and fast pointers to find the middle.
2. Reverse the second half of the Linked List.
3. Compare the first half with the reversed second half.
4. If all values are equal, return true.
5. Otherwise, return false.

Code:

```
class Solution {

public:

    bool isPalindrome(ListNode* head) {

        if (head == nullptr || head->next == nullptr) {

            return true;

        }

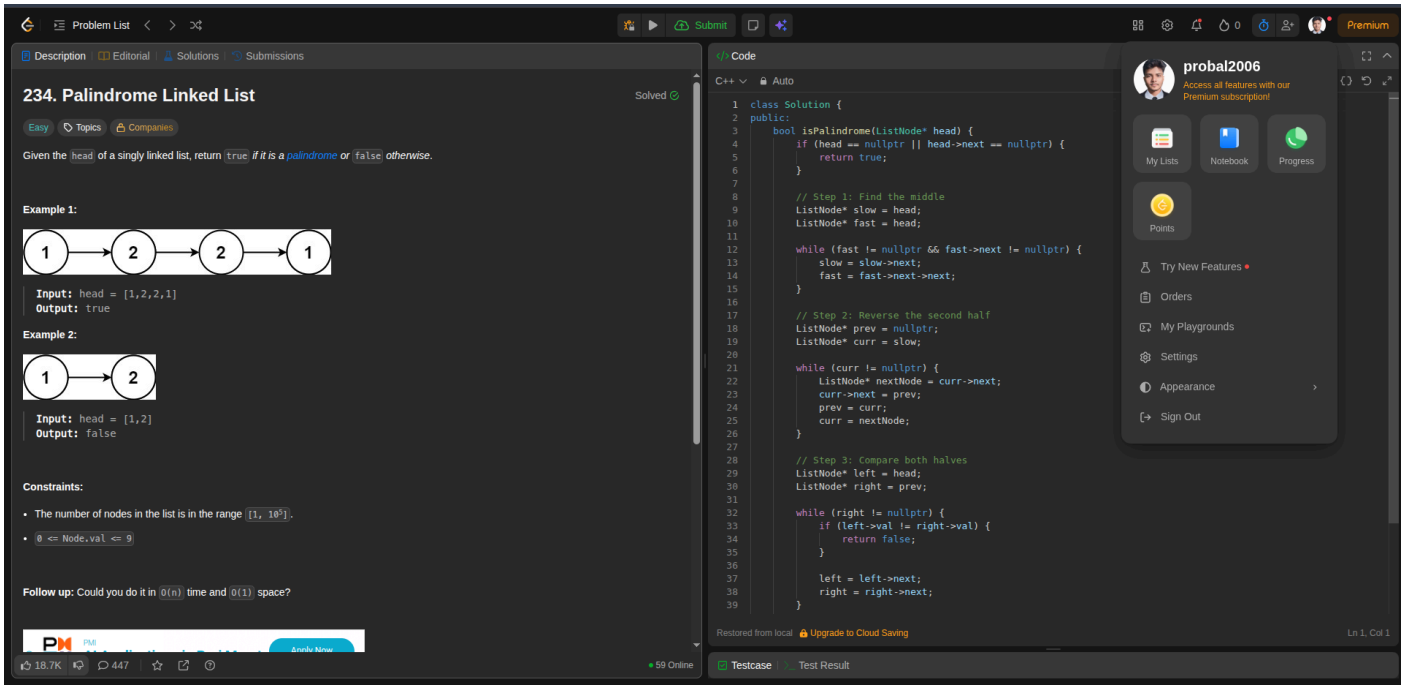
        // Step 1: Find the middle

        ListNode* slow = head;

        ListNode* fast = head;
```

```
        while (fast != nullptr && fast->next != nullptr) {
            slow = slow->next;
            fast = fast->next->next;
        }
        // Step 2: Reverse the second half
        ListNode* prev = nullptr;
        ListNode* curr = slow;
        while (curr != nullptr) {
            ListNode* nextNode = curr->next;
            curr->next = prev;
            prev = curr;
            curr = nextNode;
        }
        // Step 3: Compare both halves
        ListNode* left = head;
        ListNode* right = prev;
        while (right != nullptr) {
            if (left->val != right->val) {
                return false;
            }
            left = left->next;
            right = right->next;
        }
        return true;
    }
};
```

Output:



Observations/Outcome: The program successfully checks whether the Linked List is a palindrome by finding the middle, reversing the second half, and comparing both halves.

Time Complexity: O(n)

Space Complexity: O(1)

Learning outcomes:

1. Learned how to traverse a Linked List using pointers.
2. Learned the slow and fast pointer technique.
3. Learned how to reverse a Linked List.
4. Learned how to compare two parts of a Linked List.
5. Learned to solve Linked List problems efficiently with O(n)

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Discover. Learn. Empower.

Experiment No: 4.2

Aim:

To rearrange a singly linked list so that all odd-positioned nodes come before all even-positioned nodes.

Objective:

1. To understand Linked List node rearrangement.
2. To separate odd- and even-positioned nodes.
3. To reconnect the nodes correctly.
4. To solve the problem efficiently without extra space.

Problem Statement: Given the head of a singly linked list, group all nodes with odd indices together, followed by nodes with even indices, and return the reordered linked list.

LeetCode Link: https://leetcode.com/problems/odd-even-linked-list/?utm_source=chatgpt.com

Procedure/Algorithm/Pseudocode:

1. Store the first even node.
2. Traverse the list using odd and even pointers.
3. Connect odd nodes.
4. Connect even nodes.
5. Attach the even list after the odd list.

Code:

```
class Solution {

public:

    ListNode* oddEvenList(ListNode* head) {

        if (head == nullptr || head->next == nullptr) {

            return head;

        }

        ListNode* odd = head;

        ListNode* even = head->next;

        ListNode* evenHead = even;

        while (even != nullptr && even->next != nullptr) {

            // Connect odd nodes

            odd->next = even->next;

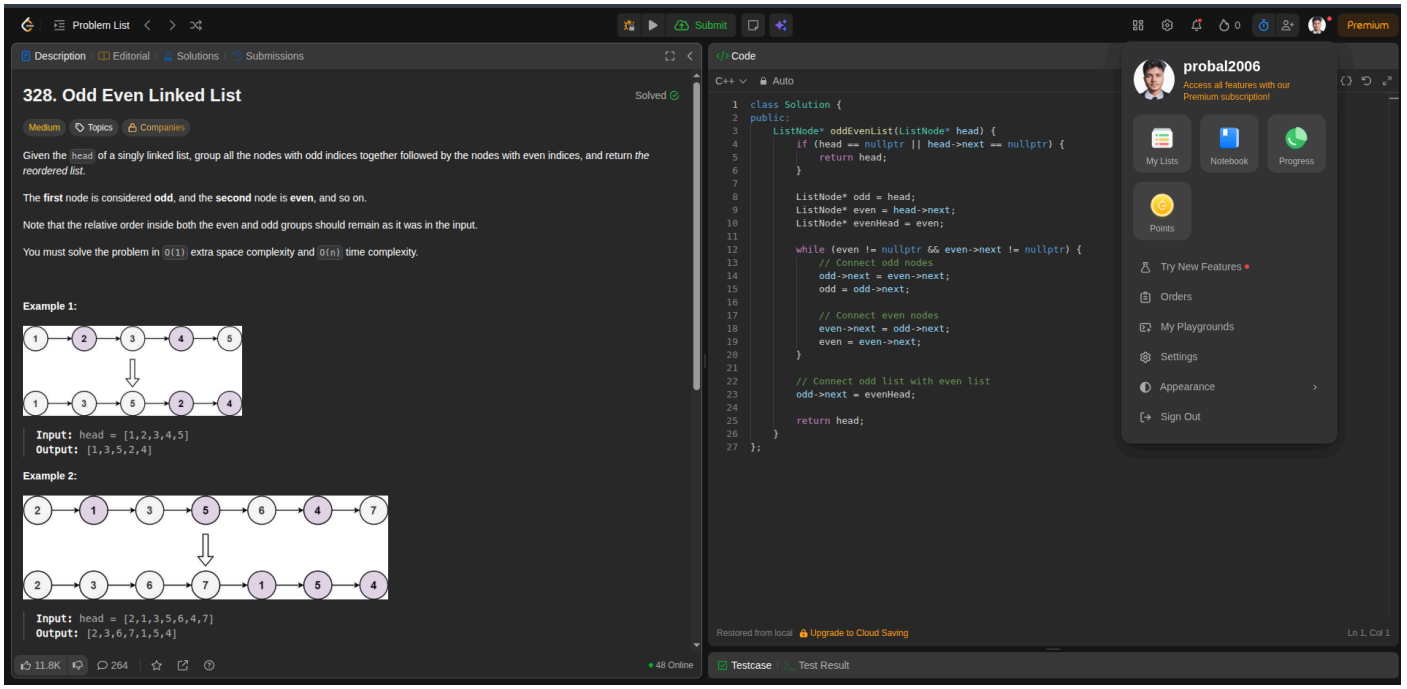
            odd = odd->next;

            // Connect even nodes

            even->next = odd->next;
```

```
            even = even->next;
        }
        // Connect odd list with even list
        odd->next = evenHead;
        return head;
    }
};
```

Output:



Observations/Outcome: The program successfully rearranges the Linked List by placing odd-positioned nodes first and even-positioned nodes after them.

Time Complexity: O(n)

Space Complexity: O(1)

Learning outcomes:

1. Learned how to rearrange nodes in a Linked List.
2. Learned to use multiple pointers.
3. Learned the difference between node position and node value.
4. Learned how to separate and reconnect Linked Lists.
5. Improved understanding of in-place Linked List operations.